

File Handling in C++

File handling means reading from and writing to files (like .txt, .csv, etc.) using classes provided by the C++ standard library.

- Programs run in RAM, meaning data exists only while the program is running, when a program ends, all data in RAM is lost automatically.
- File handling allows storing data in secondary memory (like HDD or SSD), so it can be preserved permanently and can be saved and accessed even after the program terminates.
- For file operations, C++ provides file stream classes in the `<fstream>` header such as ofstream, ifstream, fstream.

Opening a File

Before reading from or writing to a file, we first need to open it. Opening a file loads that file in the RAM. In C++, we open a file by creating a stream to it using the **fstream** class that represent the file stream i.e. stream for input and output to the file.

```
fstream str("filename.ext", mode);
```

where,

- **str**: Name given to the stream
- **filename**: Name of the file
- **mode**: Represents the way in which we are going to interact with the file.

File Opening Modes

File opening mode indicate file is opened for reading, writing, or appending. Below is the list of all file modes in C++:

Mode	Description
ios::in	File open for reading. If file does not exists, the open operation fails.
ios::out	File open for writing: the internal stream buffer supports output operations.
ios::binary	Operations are performed in binary mode rather than text.

Mode	Description
ios::ate	The output position starts at the end of the file.
ios::app	All output operations happen at the end of the file, appending to its existing contents.
ios::trunc	Any contents that existed in the file before it is open are discarded.

For Example, if we want to open the file for reading, we use the following opening mode:

```
fstream filein("file.txt", ios::in);
```

Similarly, if we want to open the file for writing, we use the following:

```
fstream fileout("file.txt", ios::out);
```

✓ These modes can also be combined using OR operator (`|`). For example, you can open the file stream in both read and write mode as shown:

✓ `fstream str("file.txt", ios::in | ios::out);`

If the file opened in write mode does not exists, a new file is created. But if the file opened in read mode doesn't exists, then no new file is created, and an exception is thrown

Other File Streams

fstream is not the only file stream provided by C++. There are two more specialized streams:

✓ **ifstream**: Stands for input file stream. It is equivalent to open fstream in **ios::in** mode.

✓ **ofstream**: Stands for output file stream. It is equivalent to opening fstream in **ios::out** mode.

The above modes are default modes for these streams. These modes cannot be changed but can be clubbed together with other modes. Now for input, we can also use `ifstream` as shown:

```
ifstream filein("file.txt");
```

Similarly, for output:

```
ofstream fileout("file.txt");
```

Write Data to File

Once the file is opened in the write mode using either **fstream** or **ofstream**, we can perform the write operation in similar way as with cout using **<< operator**.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    // Open a file
```

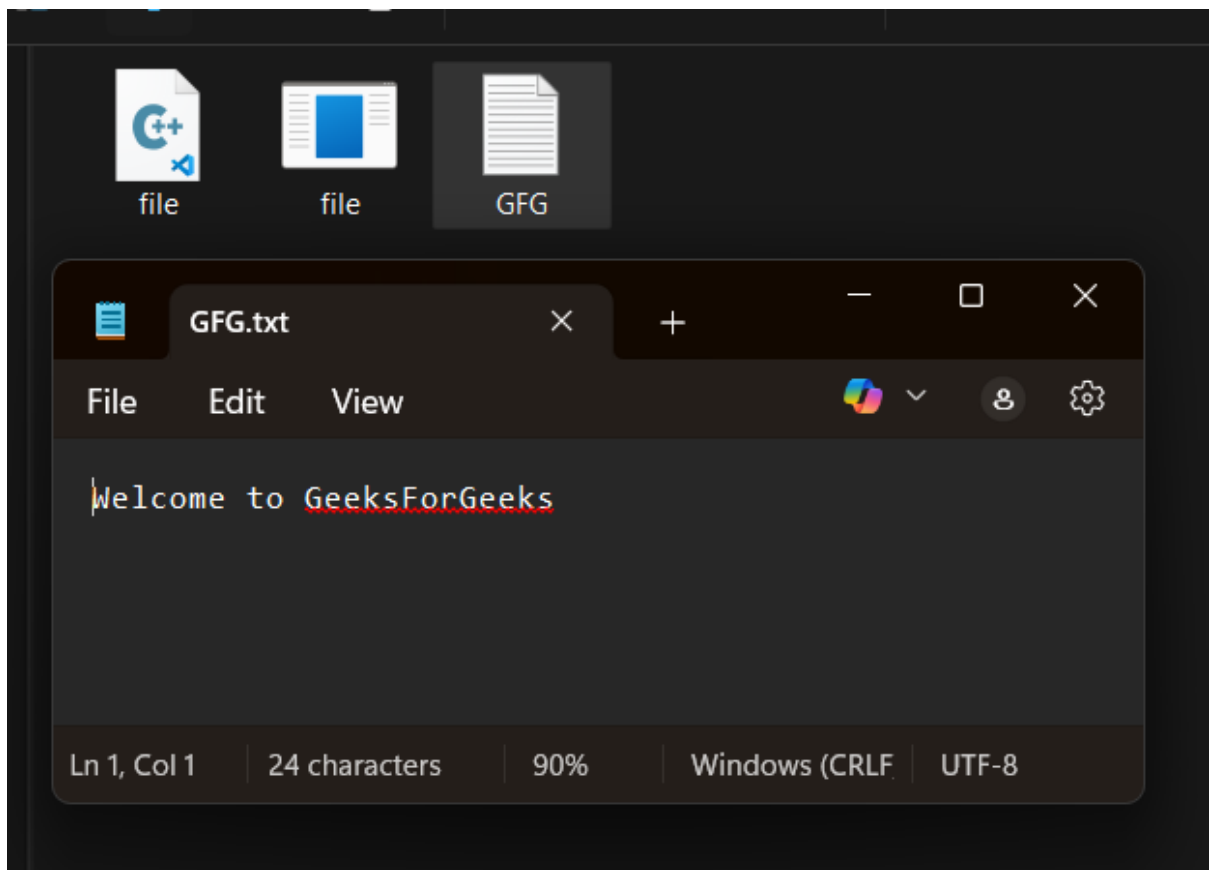
```
    ofstream file("GFG.txt");
```

```
    // Write the string to the file
```

```
    file << "Welcome to GeeksforGeeks.";
```

```
    return 0;
```

```
}
```



GFG.txt

Read Data from File

Once the file is opened in the read mode using either fstream or ifstream, we can perform the write operation in similar way as with cin using >> operator.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Open a file in read mode
```

```
    ifstream file("GFG.txt");
```

```
    string s;
```

```
    // Read string from the file
```

```
file >> s;

cout << "Read String: " << s;

return 0;
}
```

Output

Read String: Welcome

This has same problem as cin. The input is only taken until the first whitespace character. To avoid this, we can use the **getline()** function as shown:

```
#include <bits/stdc++.h>

using namespace std;

int main()
{

    // Open a file in read mode
    ifstream file("GFG.txt");

    string s;

    // Read string from the file
    getline(file, s);

    cout << "Read String: " << s;

    return 0;
}
```

in place of
cin << name;
we can use
getline(cin, name);

Output

Read String: Welcome to GeeksforGeeks.

Closing the File

Closing the file means closing the associated stream and free the resources that we being used. It is important to close the file after you are done with it, especially in the long running programs to avoid memory leaks, data loss, etc.

In C++, the files are closed using the **close()** member function that is present in all file streams.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Open a file in read mode
```

```
    ifstream file("GFG.txt");
```

```
    string s;
```

```
    // Read string from the file
```

```
    getline(file, s);
```

```
    cout << "Read String: " << s;
```

```
    // Close the file
```

```
    file.close();
```

```
    return 0;
```

```
}
```

Output

Read String: Welcome to GeeksforGeeks.

Errors in File Handling

Many different types of errors can occur in file handling such as file not found, disk full, etc. Our programs should expect common errors and should be able to handle them properly. Following are some common errors that can occur during file handling:

File Open Failure

There can be cases in which the file is not opened due to various reasons such as it doesn't exist, or the program does not have permission to open it, etc. In this case, we can use the **is_open()** member function of the file stream classes to check whether the file is opened successfully or not.

```
#include <bits/stdc++.h>

using namespace std;

int main() {

    fstream file("nonexistent_file.txt", ios::in);

    // Check if the file is opened
    if (!file.is_open()) {

        cerr << "Error: Unable to open file!" << endl;

        return 1;

    }

    file.close();

    return 0;

}
```

Output

Error: Unable to open file!

Failure to Read/Write Data

Another common error is failure to read or write data for reasons such as incorrect mode, etc. In this case, we can validate operations after each read/write attempt. For example, reading using `getline()` can be validated as shows:

```
#include <bits/stdc++.h>

using namespace std;

int main() {

    fstream file("GFG.txt", ios::out);

    if (!file.is_open()) {

        cerr << "Error: Unable to open file!" << endl;

        return 1;

    }

    string line;

    // Checking if getline() read successfully or not
    if (!getline(file, line))
        cerr << "Error: Failed to read data" << endl;

    file.close();

    return 0;

}
```

Output

Error: Failed to read data

End-of-File (EOF) Error

Trying to read beyond the end of the file can cause an EOF error. This can happen when you don't check for the end of the file before reading. We can check for EOF using **eof()** member function.

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    ifstream file("GFG.txt");

    if (!file.is_open())
    {
        cerr << "Error: Unable to open file!" << endl;
        return 1;
    }

    string line;
    while (getline(file, line))
        cout << line << endl;

    // Check for eof
    if (file.eof())
        cout << "Reached end of file." << endl;
    else
        cerr << "Error: File reading failed!" << endl;

    file.close();

    return 0;
}
```

Output

Reached end of file.

Notice that we have also validated the read operation before checking EOF as **getline()** will only return **nullptr** even if the read fails due to any reason.

Handling Binary Files

In C++, we can also handle **binary files**, which store data in raw format. To read and write binary data, must use the **ios::binary** flag when creating/opening a binary file.

Write into Binary File

To write data to a binary file, we first need to open or create the file in **ios::binary** mode.

```
#include <cstring>

#include <fstream>

#include <iostream>

using namespace std;

int main()
{
    string str = "Welcome to GeeksForGeeks";

    // Open a binary file for writing
    ofstream file("fileBin.bin", ios::binary);

    // Check if the file is open
    if (!file)
    {
        cerr << "Error opening the file for writing.";
        return 1;
    }
```

```

// Write the length of the string (size) to file first
size_t strLength = str.length();
file.write(reinterpret_cast<const char *>(&strLength), sizeof(strLength));

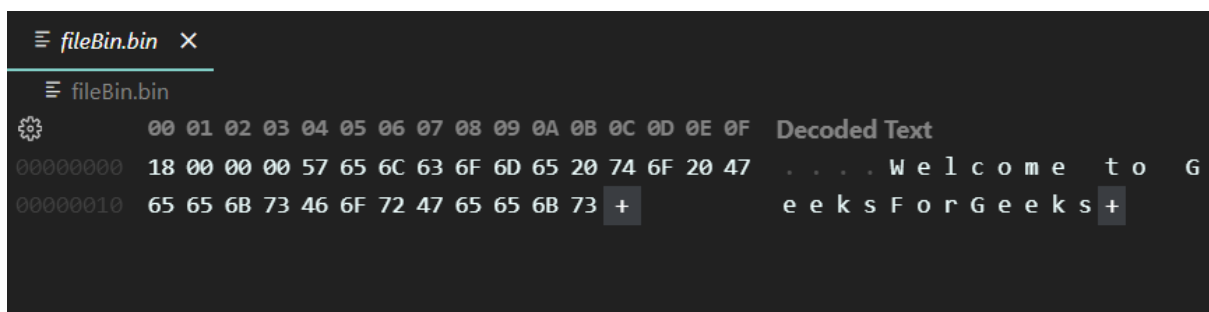
// Write the string to the binary file
file.write(str.c_str(), strLength);

// Close the file
file.close();

return 0;
}

```

Output



Binary File

Reading from Binary File

Just as we open a file in binary mode to write data, to read data from a binary file, we must open the file in read mode using **ios::in**.

Syntax:

```

fstream fileInstance("fileName.bin", ios::in | ios::binary);

#include <cstring>

#include <fstream>

#include <iostream>

using namespace std;

```

```
int main()
{
    string str;

    // Open the binary file for reading
    fstream file("fileBin.bin", ios::in | ios::binary);

    // Check if the file is open
    if (!file)
    {
        cerr << "Error opening the file for reading.";
        return 1;
    }

    // Read the length of the string (size) from the file
    size_t strLength;
    file.read(reinterpret_cast<char *>(&strLength), sizeof(strLength));

    // Allocate memory for the string and read the data
    char *buffer = new char[strLength + 1]; // +1 for the null-terminator
    file.read(buffer, strLength);

    // Null-terminate the string
    buffer[strLength] = '\0';

    // Convert the buffer to a string
    str = buffer;
```

```

// Print file data

cout << "File Data: " << str;

delete[] buffer;

file.close();

return 0;
}

```

Output

File Data: Welcome to GeeksForGeeks

How to Delete a File in C++?

C++ file handling allows us to manipulate external files from our C++ program. We can create, remove, and update files using file handling. In this article, we will learn how to remove a file in C++.

Delete a File in C++

To remove a file in C++, we can use the [remove\(\)](#) function defined inside the `<stdio.h>` header file. The `remove()` function takes the path of the file as a string argument and deletes the file.

Syntax of remove()

```
remove(char * path)
```

where the path is the relative or even absolute path to the file.

If the function returns zero, the file has been successfully deleted. If the function returns a non-zero value, an error occurs.

C++ Program to Remove a File

The below example demonstrates how we can remove a file named “myfile.txt” in C++.

```
// C++ program to demonstrate how to remove a file
```

```
#include <stdio.h>
```

```

#include <iostream>

using namespace std;

int main()
{
    // Remove the file named "myfile.txt"
    int status = remove("myfile.txt");

    // Check if the file has been successfully removed
    if (status != 0) {
        perror("Error deleting file");
    }
    else {
        cout << "File successfully deleted" << endl;
    }

    return 0;
}

```

Output

File successfully deleted

C++ Program to Append a String in an Existing File

Here, we will build a C++ program to append a string in an existing file using 2 approaches i.e.

1. Using **ofstream**
2. Using **fstream**

C++ programming language offers a library called *fstream* consisting of different kinds of classes to handle the files while working on them. The classes present in *fstream* are *ofstream*, *ifstream* and *fstream*.

The file we are considering the below examples consists of the text "*Geeks for Geeks*".

1. Using "*ofstream*"

In the below code we appended a string to the "Geeks for Geeks.txt" file and printed the data in the file after appending the text. The created ofstream "*ofstream of*" specifies the file to be opened in write mode and "*ios::app*" in the open method specifies the append mode.

```
// C++ program to demonstrate appending of
// a string using ofstream

#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main()
{
    ofstream of;
    fstream f;

    // opening file using ofstream
    of.open("Geeks for Geeks.txt", ios::app);
    if (!of)
        cout << "No such file found";
    else {
        of << " String";
        cout << "Data appended successfully\n";
        of.close();
        string word;

        // opening file using fstream
        f.open("Geeks for Geeks.txt");
        while (f >> word) {
            cout << word << " ";
```

```

    }
    f.close();
}
return 0;
}

```

Output:

Data appended successfully

Geeks for Geeks String

2. Using "*fstream*"

In the below code we appended a string to the "*Geeks for Geeks.txt*" file and printed the data in the file after appending the text. The created *fstream* "*fstream f*" specifies the file to be opened in reading & writing mode and "*ios::app*" in the open method specifies the append mode.

```
// C++ program to demonstrate appending of
```

```
// a string using fstream
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    fstream f;
```

```
    f.open("Geeks for Geeks.txt", ios::app);
```

```
    if (!f)
```

```
        cout << "No such file found";
```

```
    else {
```

```
        f << " String_fstream";
```

```
        cout << "Data appended successfully\n";
```

```
        f.close();
```



```

    string word;

    f.open("Geeks for Geeks.txt");

    while (f >> word) {

        cout << word << " ";

    }

    f.close();

}

return 0;

}

```

Output:

Data appended successfully

Geeks for Geeks String_fstream

C++ Program to Copy One File into Another File

To copy the text/contents of one file to another file, we should know the basics of reading and writing a text file in C++. To copy the file using C++, we read the contents of the source file and write it into the destination file.

Steps to copy one file to another in C++:

1. Create objects of ifstream and ofstream classes.
2. Check if they are connected to their respective files. If so, go ahead otherwise check the filenames twice.
3. Read the contents of the source file using the getline() method and write the same to the destination using the << operator (i.e. copy each line from ifstream object to ofstream object).
4. Close files after the copy using the close() method.
5. End the program.

Note: ifstream and ofstream classes are present in the <fstream> library.

Example:

// C++ Program to demonstrate

// copying the content of a .txt file

```
#include <fstream>

#include <iostream>

using namespace std;

int main()
{
    string line;

    // For writing text file

    // Creating ofstream & ifstream class object
    ifstream ini_file{
        "original.txt"
    }; // This is the original file
    ofstream out_file{ "copy.txt" };

    if (ini_file && out_file) {

        while (getline(ini_file, line)) {
            out_file << line << "\n";
        }

        cout << "Copy Finished \n";
    }

    else {
        // Something went wrong
        printf("Cannot read File");
    }

    // Closing file
    ini_file.close();
    out_file.close();

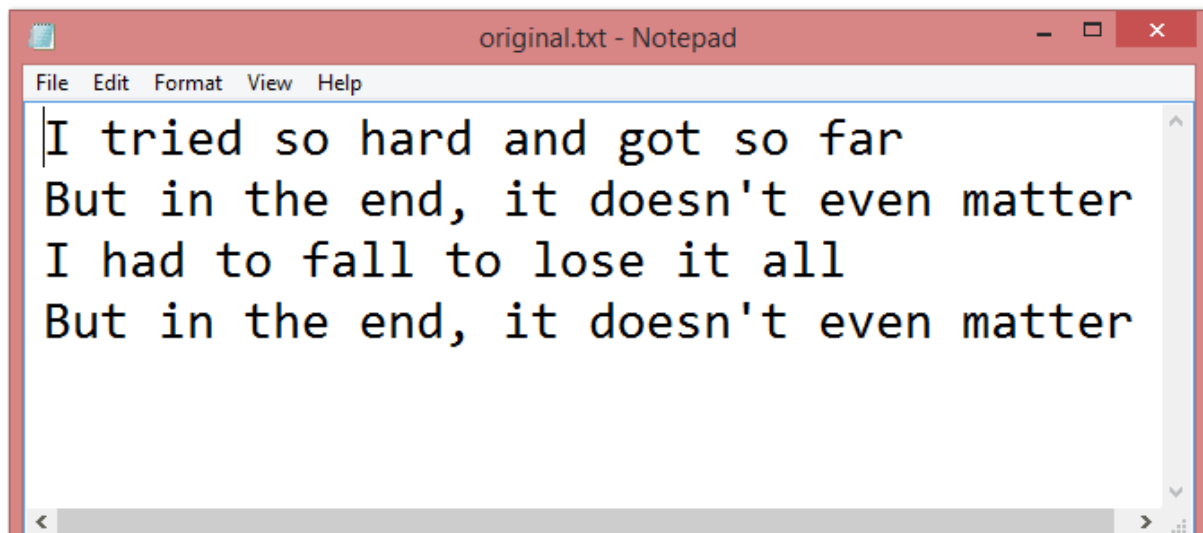
    return 0;
}
```

```
}
```

Output:

Original File – original.txt:

Name	Date modified	Type	Size
bin	12/27/2018 12:59 ...	File folder	
obj	12/27/2018 12:59 ...	File folder	
Copy Files C++.cbp	12/27/2018 12:30 ...	CBP File	2 KB
Copy Files C++.depend	12/27/2018 2:23 PM	DEPEND File	1 KB
Copy Files C++.layout	12/27/2018 1:16 PM	LAYOUT File	1 KB
main.cpp	12/27/2018 2:21 PM	CPP File	1 KB
original.txt	12/27/2018 1:01 PM	Text Document	1 KB



```
original.txt - Notepad
File Edit Format View Help
I tried so hard and got so far
But in the end, it doesn't even matter
I had to fall to lose it all
But in the end, it doesn't even matter
```

File: original.txt

Copy program running:

A screenshot of a Windows command prompt window. The title bar reads "C:\Users\Win8.1\Saved Games\Copy Files C++\bin\Debug\Copy Files C++.exe". The window content shows the text "Copy Finished", "Process returned 0 (0x0) execution time : 0.046 s", and "Press any key to continue.".

Copy File – copy.txt:

Name	Date modified	Type	Size
bin	12/27/2018 12:59 ...	File folder	
obj	12/27/2018 12:59 ...	File folder	
Copy Files C++.cbp	12/27/2018 12:30 ...	CBP File	2 KB
Copy Files C++.depend	12/27/2018 2:23 PM	DEPEND File	1 KB
Copy Files C++.layout	12/27/2018 1:16 PM	LAYOUT File	1 KB
copy.txt	12/27/2018 2:25 PM	Text Document	1 KB
main.cpp	12/27/2018 2:21 PM	CPP File	1 KB
original.txt	12/27/2018 1:01 PM	Text Document	1 KB

A screenshot of a Notepad window titled "copy.txt - Notepad". The window contains the text:

I tried so hard and got so far
But in the end, it doesn't even matter
I had to fall to lose it all
But in the end, it doesn't even matter

Copied file: copy.txt

In this example, we have assumed that both the original file and the copy file are in the same directory where the code file of this program is. The above program runs unless the whole contents of the original file get copied to another file.